

2 - Feature Detection

Feature Detection, Optical Flow, Fitting and Alignment

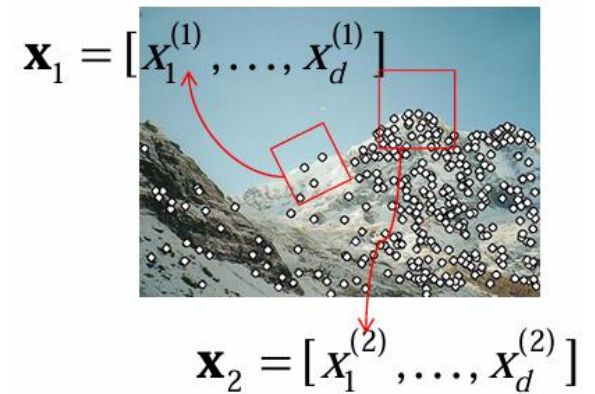
Correspondence

- **Correspondence** is about matching features such as points, edges, corners, or regions across multiple views or images.

1. Identify **feature points** / **keypoints** / **points of interest**.

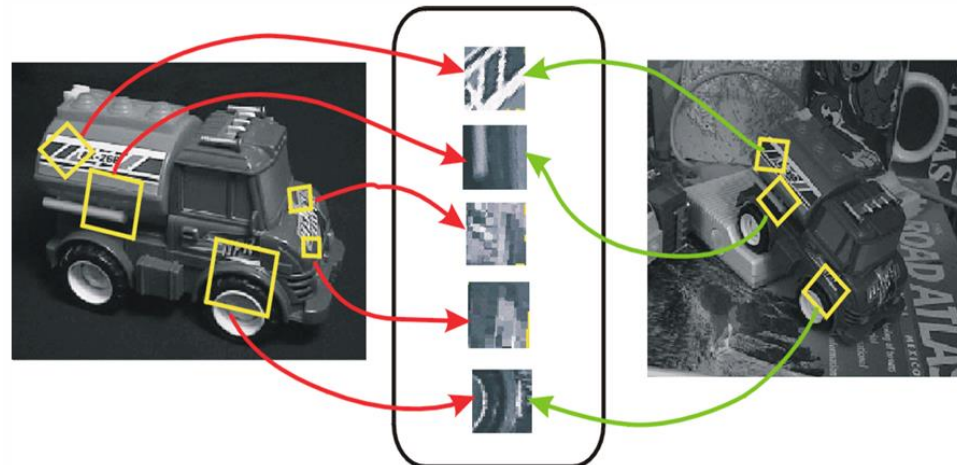
2. Extract a **feature descriptor** (vector) around each keypoint.

3. Find correspondence in pairs between the descriptors of different views.



What makes a good feature?

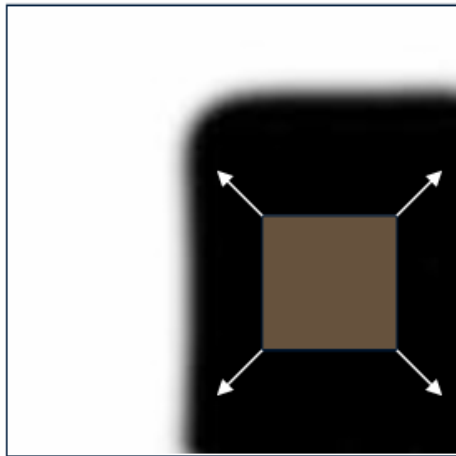
- **Repeatability:** under geometric and photometric transformations, the same feature can be found in different images.
- **Saliency:** every feature is as unique and distinctive as possible.
- **Efficiency:** features should not be too many, but also not too few.
- **Locality:** features should be small enough to handle occlusion and clutter, but also large enough to have a distinctive descriptor.



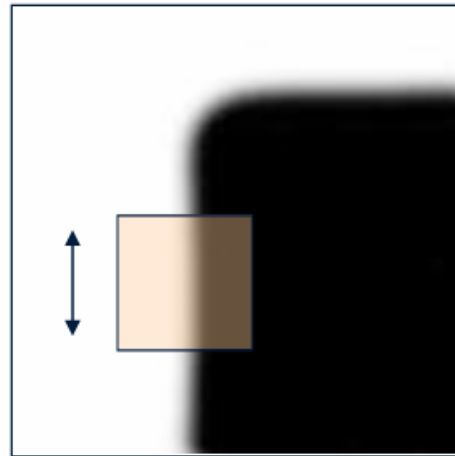
Feature Descriptors

Corner Detection

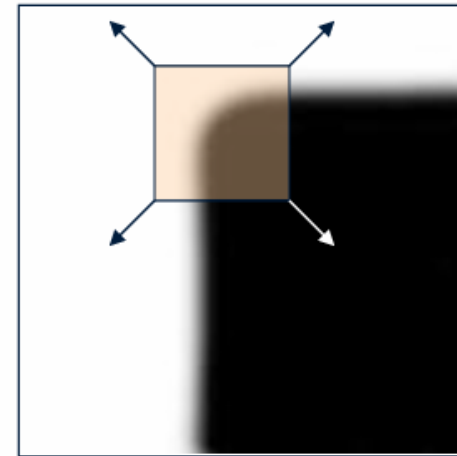
- Corners are repeatable and distinctive features.
- Detecting corners is done by shifting a small window towards any direction and measuring a large change in e.g., intensity in the region below it.



“flat” region:
no change in
all directions



“edge”:
no change
along the edge
direction



“corner”:
significant
change in all
directions

Corner Detection

- Change in appearance of window $w(x, y)$ for the shift (u, v) :

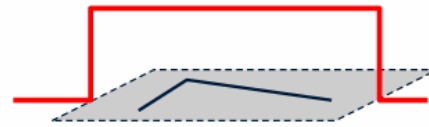
$$E(u, v) = \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

Window
function

Shifted
intensity

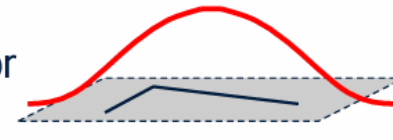
Intensity

Window function $w(x, y) =$



1 in window, 0 outside

or



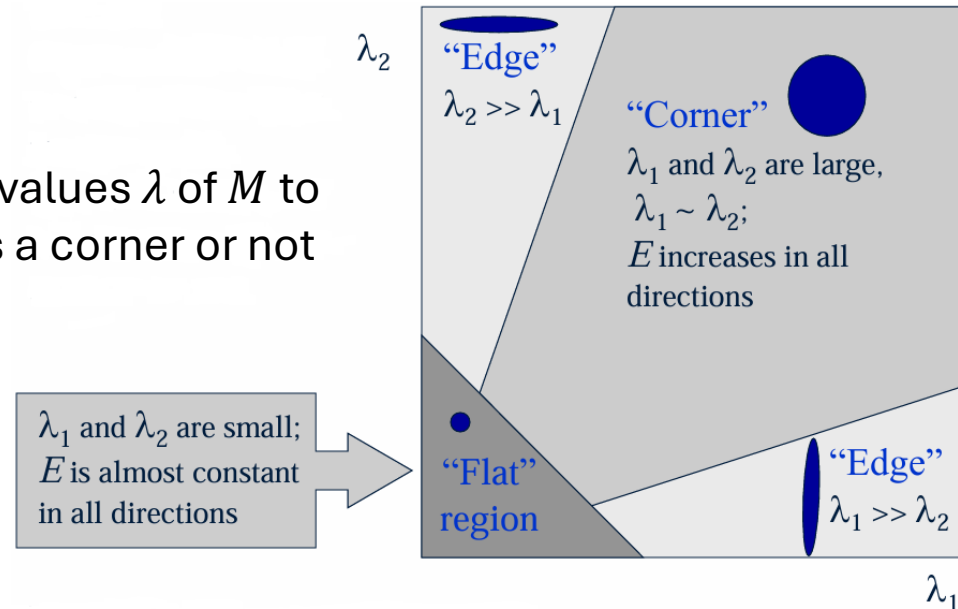
Gaussian

- We want to find patches with a large $E(u, v)$.

Corner Detection

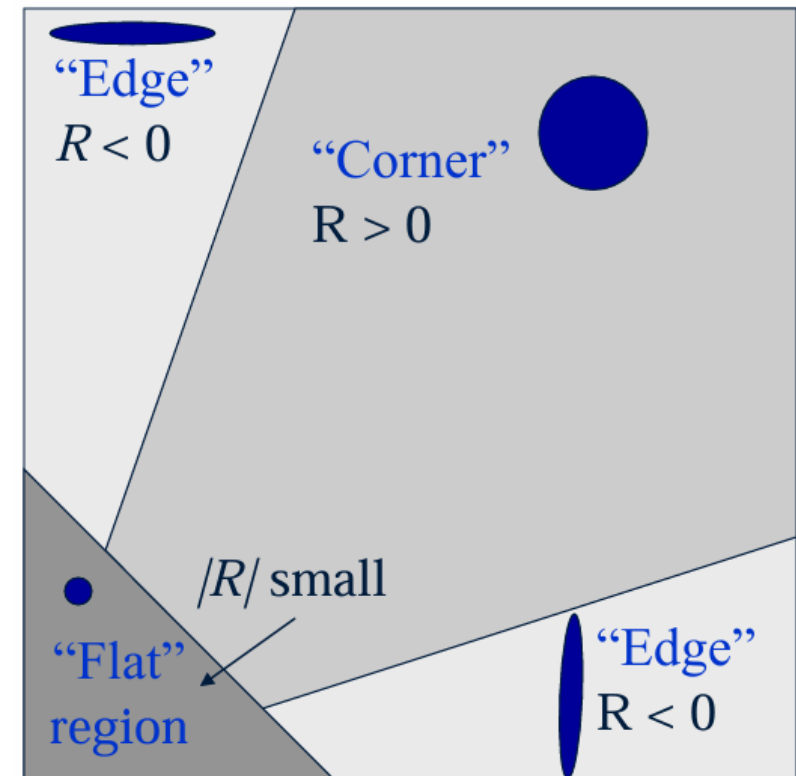
- $E(u, v) \approx [u \quad v] M \begin{bmatrix} u \\ v \end{bmatrix}$, where M is a *second moment matrix* computed from image derivatives: $M = \sum_{x,y} w(x, y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$, where $I_x = \frac{\partial I}{\partial x}$ and $I_y = \frac{\partial I}{\partial y}$.
- M is symmetrical and can be diagonalized to $M = R^{-1} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} R$.

We can use the eigenvalues λ of M to determine if a point is a corner or not



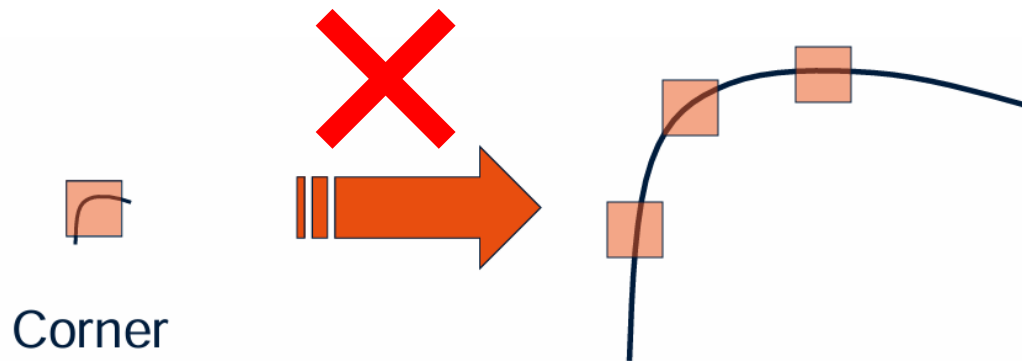
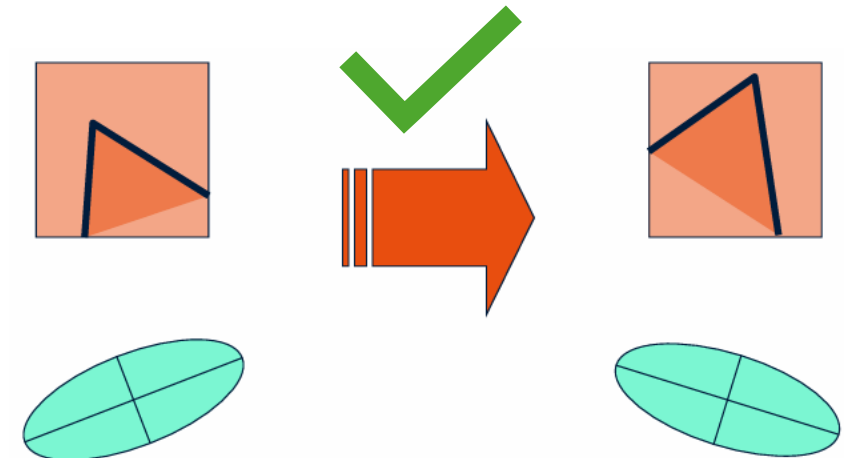
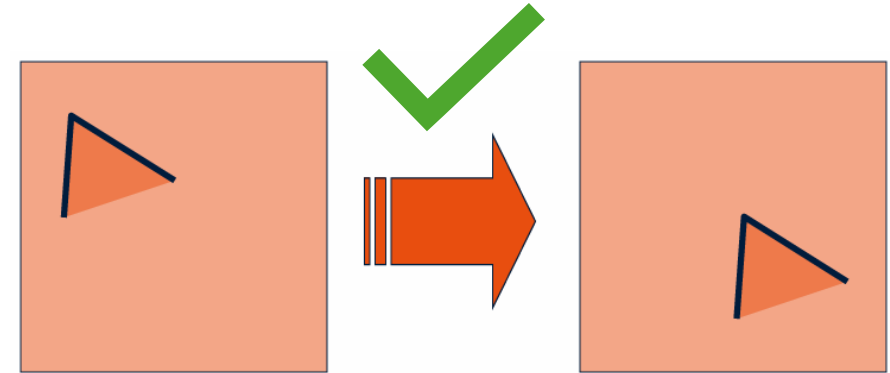
Corner Detection - Harris Corner Detection

1. For each window, compute the corresponding M matrix.
2. Get a 'cornerness score' for each window: $R = \lambda_1 \lambda_2 - \alpha(\lambda_1 + \lambda_2)^2$.
3. Choose windows with R above a certain threshold.
4. Perform **non-maximum suppression**.



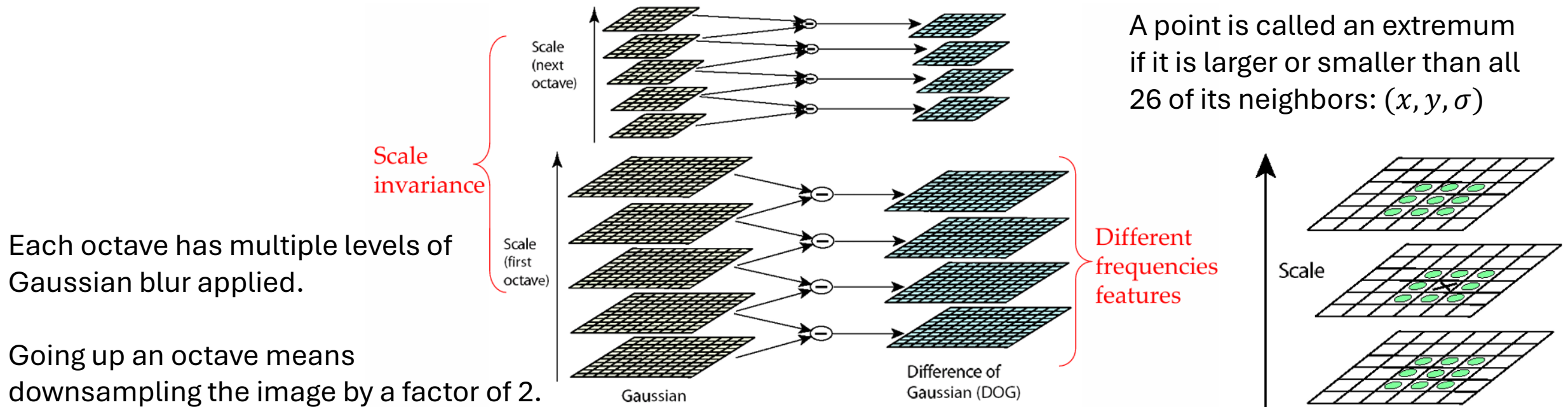
Corner Detection - Harris Corner Detection

- Harris corner detection is:
 - Invariant to intensity shifts.
 - *Subject to intensity scaling.*
 - Invariant to image translation.
 - Invariant to image rotation.
 - *Subject to image scaling.*

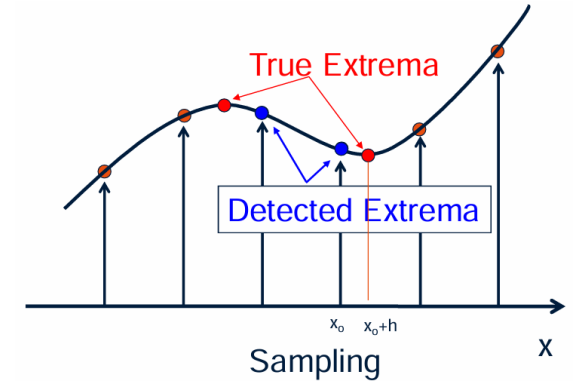


SIFT - Feature Detection

- Most information about an object lies on its edges, so **Scale Invariant Feature Transform (SIFT)** looks for the most informative edge points, called *extrema*.
- Extrema are found by constructing a **Gaussian- and DoG Pyramid**:



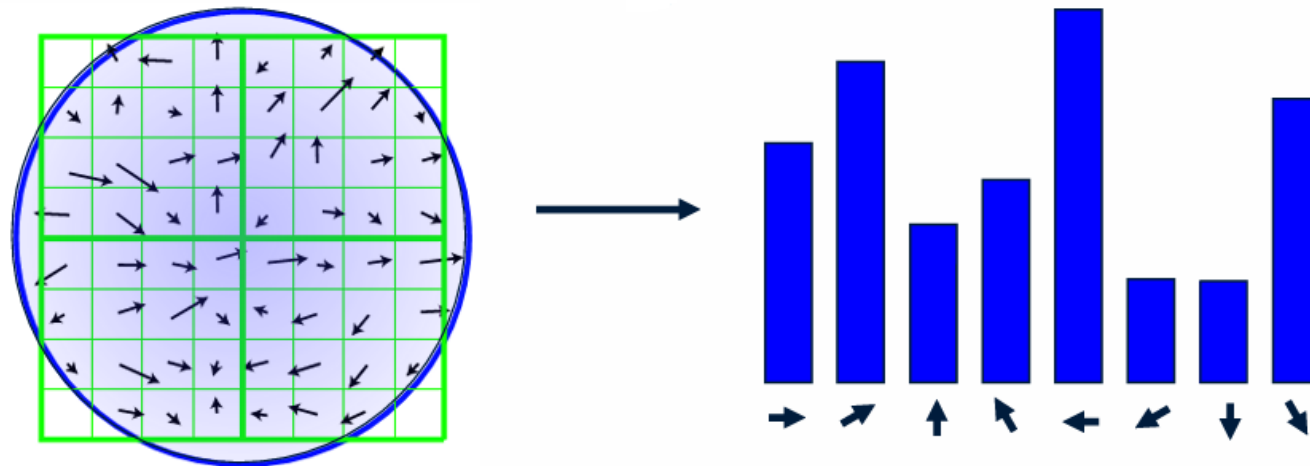
SIFT - Feature Detection



- Not all sampled keypoints lie exactly at the true extrema.
- By expressing the DoG around a keypoint through a 2nd order Taylor series expansion, we can identify true extrema in this local neighborhood.
- Discarding points that lie further than a certain distance away from true extrema is called **minimum contrast thresholding**.
- Another problem is that some keypoints are very sensitive to simple transformations and not that distinctive.
- To make the keypoints more robust, SIFT also applies **cornerness score thresholding**, similar to Harris corner detection.

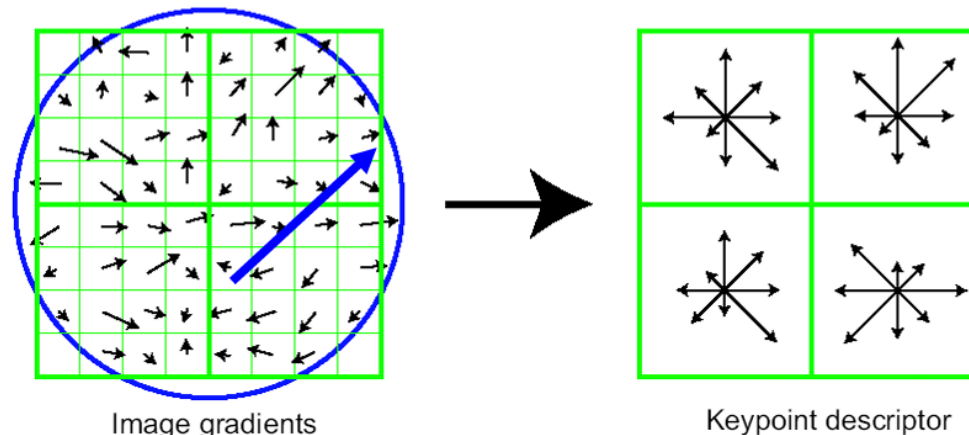
SIFT - Feature Descriptor

- The SIFT feature descriptor starts by computing the gradient's *magnitude* and *orientation* in a neighborhood around each keypoint.
- For each neighborhood, a histogram of the gradient orientation is created with 36 bins, weighted by the magnitude of the gradient.
- The keypoint is assigned the orientation at the top of the histogram.
- This gives every keypoint a *location*, *scale*, and *orientation*: (x, y, σ, θ) .



SIFT - Feature Descriptor

- The SIFT feature descriptor for each keypoint is computed as follows:
 1. Select a 16x16 area around the keypoint, rotate and scale it to match the keypoint's scale and orientation.
 2. Create gradient orientation histograms on 4x4 patches with 8 bins each, weighted by gradient magnitude.
 3. Turn the values of the histograms into a 128-dimensional vector, normalize to unit length.

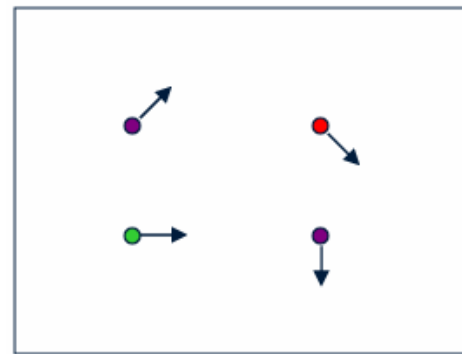


SIFT - Feature Descriptor

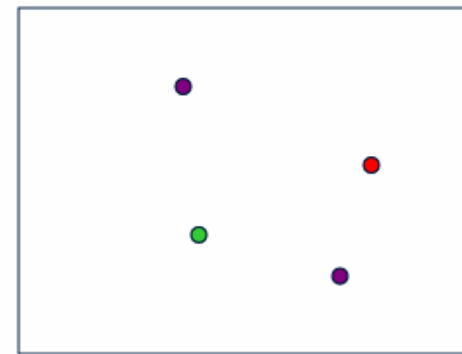
- SIFT features are robust to scale, rotation, illumination, and transformations.
- To reduce the effect of *non-linear illumination*: apply a threshold of 0.2, then re-normalize.
- SIFT is used for *training object recognition models*: extract SIFT keypoints, then create a database of the related descriptors.
- SIFT is also used to *query images*: extract SIFT keypoints, find matches in the database through a threshold: $\frac{\text{nearest neighbor}}{2^{\text{nd}} \text{ nearest neighbor}} < 0.8$ for each descriptor, find clusters of at least 3 keypoints matches, perform a geometric fit to verify each cluster.

Optical Flow

- How can we estimate a pixel's motion from image H to image I ? This is known as the **pixel correspondence problem**. Key assumptions:
 - **Color Constancy**: a point in H looks the same in I .
 - **Small Motion**: points do not move very far in successive frames.
- We can derive the basic equation $H(x, y) = I(x + u, y + v)$, where we are looking for (u, v) .



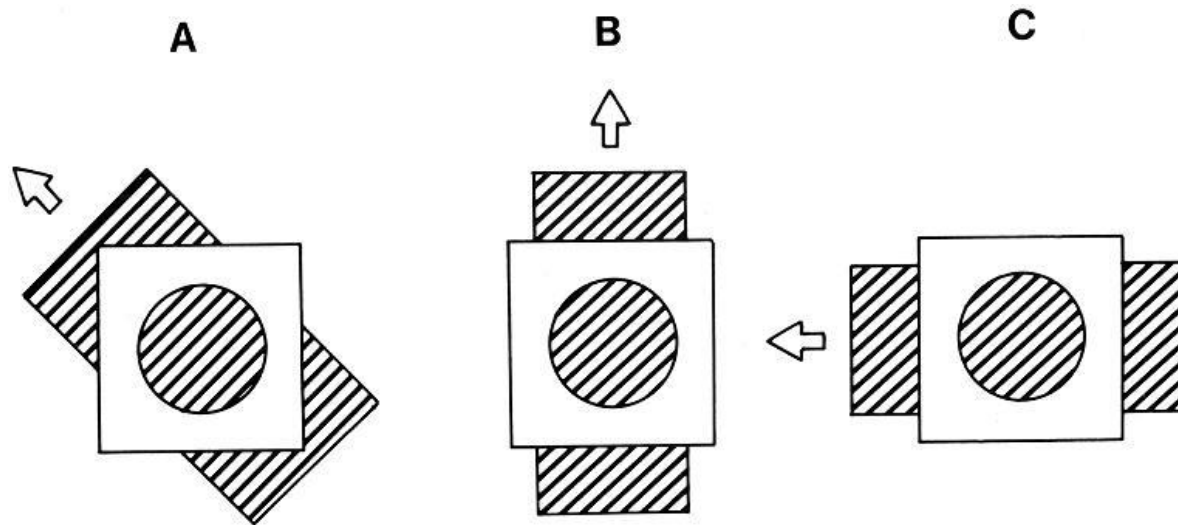
$H(x, y)$



$I(x, y)$

Optical Flow

- $H(x, y) = I(x + u, y + v) \rightarrow I_t + \nabla I \cdot [u \ v] = 0$, where I_t is the time derivative of I at (x, y) . This gives us 1 equation with 2 unknowns per pixel.
- This is known as the **aperture problem**: we know the direction of flow in the gradient direction, but the direction of flow parallel to an edge is unknown.



Optical Flow - Lukas-Kanade

- Assuming the motion is uniform in a small 5x5 window, we now get 25 equations per pixel! This is known as **Lukas-Kanade Optical Flow (L-K)**.

$$\underbrace{\begin{bmatrix} I_x(\mathbf{p}_1) & I_y(\mathbf{p}_1) \\ I_x(\mathbf{p}_2) & I_y(\mathbf{p}_2) \\ \vdots & \vdots \\ I_x(\mathbf{p}_{25}) & I_y(\mathbf{p}_{25}) \end{bmatrix}}_{\substack{A \\ 25 \times 2}} \underbrace{\begin{bmatrix} u \\ v \end{bmatrix}}_{\substack{d \\ 2 \times 1}} = - \underbrace{\begin{bmatrix} I_t(\mathbf{p}_1) \\ I_t(\mathbf{p}_2) \\ \vdots \\ I_t(\mathbf{p}_{25}) \end{bmatrix}}_{\substack{b \\ 25 \times 1}}$$

- To find the optimal (u, v) , we can solve the following equation:

$$\underbrace{\begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix}}_{A^T A} \underbrace{\begin{bmatrix} u \\ v \end{bmatrix}}_{\substack{\text{(summation over all} \\ \text{pixels in the window)}}} = - \underbrace{\begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}}_{A^T b}$$

Optical Flow - Lukas-Kanade

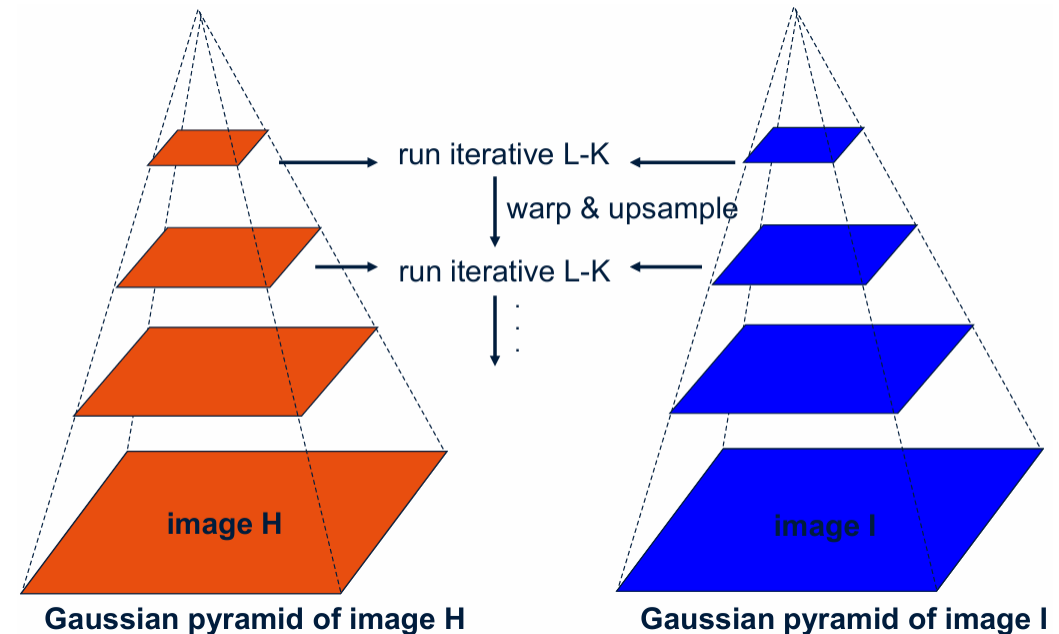
$$\underbrace{\begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix}}_{A^T A} \begin{bmatrix} u \\ v \end{bmatrix} = - \underbrace{\begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}}_{A^T b}$$

- Possible issues:
 - $A^T A$ must be invertible.
 - Eigenvalues (λ_1, λ_2) of $A^T A$ must not be too small (low textured regions don't work).
 - The ratio $\frac{\lambda_1}{\lambda_2}$ must not be too large (edges don't work).
 - Optical flow assumptions violated.
 - Motion in window is not uniform.
 - Window size is too small / large.

• If motion is not small enough:

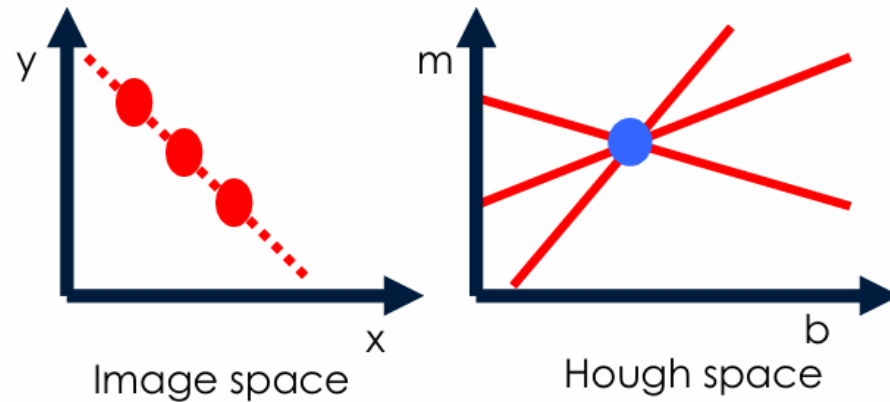
Reduce the solution!

(Coarse-to-Fine Optical Flow)



Hough Transform

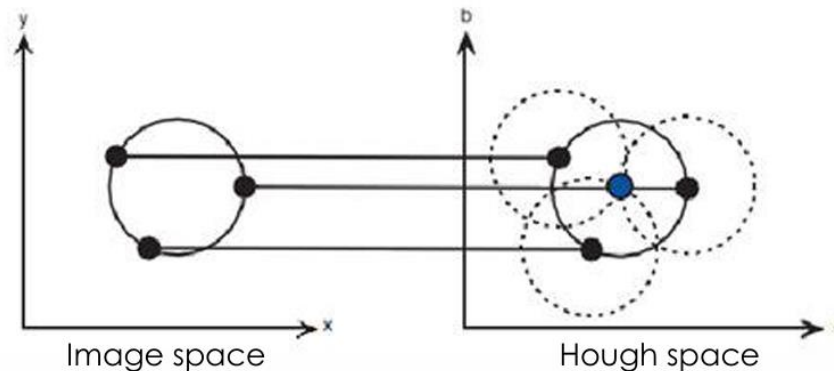
- The **Hough Transform** is a way to estimate a model that explains a set of data points in the best possible way.



- Assuming the model is of the form $y = mx + b$, each red line in the *Hough space* represents all possible values of (m, b) that fit a respective data point.
- All points *vote* for a set of parameters (robust to outliers!).

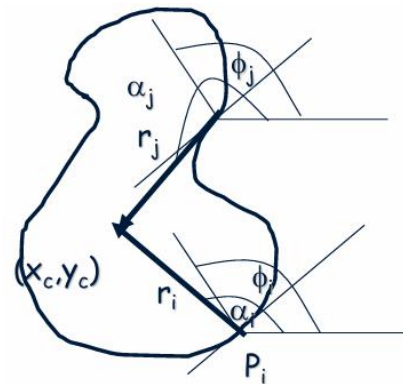
Hough Transform

- The Hough Transform can also be used to identify shapes:
 1. Determine what shape you want to find, e.g., a circle with fixed radius r .
 2. Apply an edge detector to get a binary image, e.g., canny edge detection.
 3. Map all edge pixels to the Hough space (for a circle with fixed r : (a, b)).
 4. Divide the image space into a discrete set of bins, count the number of votes per bin.
 5. Threshold to find peaks, which correspond to detected shapes.



Generalized Hough Transform

- The **Generalized Hough Transform** works for arbitrary shapes:
 1. First draw the desired shape and pick a reference point (x_c, y_c) .
 2. Quantize the shape's edge, for each point calculate:
 - The gradient orientation ϕ .
 - The direction α and length r of the vector from the edge to the reference point (x_c, y_c) .
 3. For each ϕ , store the matching (α, r) in a table called the **R-Table**.



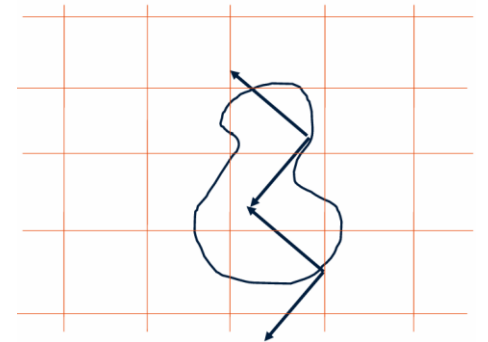
$$x_c = x_i + r_i \cos(\alpha_i)$$

$$y_c = y_i + r_i \sin(\alpha_i)$$

ϕ_1	$(r^1_1, \alpha^1_1), (r^1_2, \alpha^1_2), \dots, (r^1_{n1}, \alpha^1_{n1})$
ϕ_2	$(r^2_1, \alpha^2_1), (r^2_2, \alpha^2_2), \dots, (r^2_{n2}, \alpha^2_{n2})$
.	.
.	.
.	.
ϕ_m	$(r^m_1, \alpha^m_1), (r^m_2, \alpha^m_2), \dots, (r^m_{nm}, \alpha^m_{nm})$

Generalized Hough Transform

- When testing for the shape:
 1. Calculate the gradient orientation ϕ for each detected edge point.
 2. Look up the R-Table for that ϕ , and get all (α, r) pairs.
 3. For each (α, r) , calculate possible reference points.
 4. Vote on the reference points and threshold.
- This only works for a single scale and orientation of the shape! To take more into account, increase the dimensions of the Hough space (search space grows exponentially!).
- A similar procedure can be used to detect objects in images.



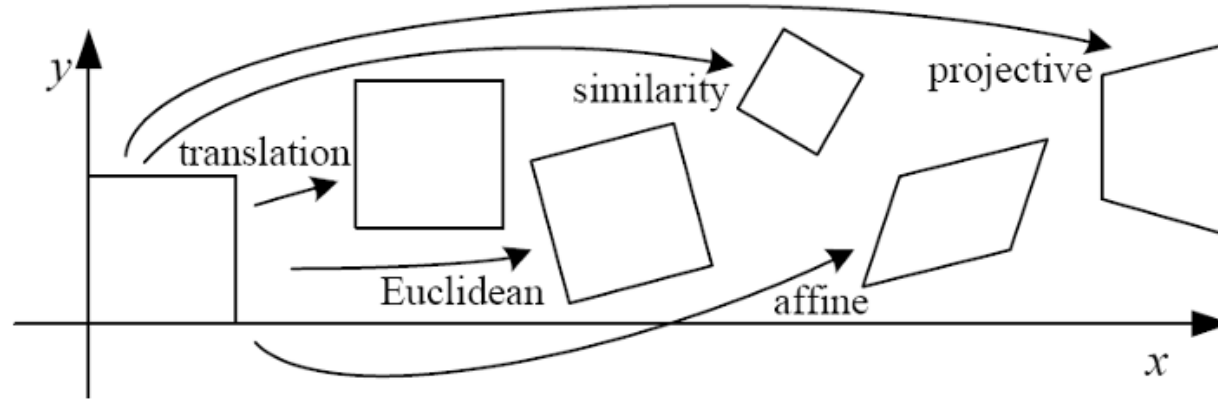
Parametric Warping

- A transformation T is a coordinate-changing function: $p' = T(p)$
- For linear transformations, we can represent T as a matrix: $\begin{bmatrix} x' \\ y' \end{bmatrix} = T \begin{bmatrix} x \\ y \end{bmatrix}$
- **Scaling:** $\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$
- **Rotation:** $\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$ ($R^{-1} = R^T$)
- **Translate:** $\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$
- **Shear:** $\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & a \\ b & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$

Affine: combination of scaling, rotation, translate, shear

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Parametric Warping



Name	Matrix	# D.O.F.	Preserves:	Icon
translation	$\begin{bmatrix} I & t \end{bmatrix}_{2 \times 3}$	2	orientation + ...	
rigid (Euclidean)	$\begin{bmatrix} R & t \end{bmatrix}_{2 \times 3}$	3	lengths + ...	
similarity	$\begin{bmatrix} sR & t \end{bmatrix}_{2 \times 3}$	4	angles + ...	
affine	$\begin{bmatrix} A \end{bmatrix}_{2 \times 3}$	6	parallelism + ...	
projective	$\begin{bmatrix} \tilde{H} \end{bmatrix}_{3 \times 3}$	8	straight lines	

RANSAC

- For N iterations:
 - Sample s pairs of keypoints from the data.
 - Fit a model on s .
 - Classify all other points as *outliers* or *inliers*.
 - If there are T or more inliers, refit the model using all inliers.
- Choose the best model (fitting error as criterion).
- $N = \frac{\log(1-p)}{\log(1-(1-e)^s)}$, where e is the probability of sampling an outlier, s is the no. points needed to fit a model, p is the desired probability of success.
- Early termination is possible when the inlier-ratio reaches its expected value:
 $(1 - e) \cdot \text{total_number_of_pairs}$

Compared to Hough transform:

- Can handle more parameters
- Worse computational time

Iterative Closest Points

- **Iterative Closest Points (ITC)** is an algorithm that can be used to fit a transformation when no prior-matched pairs are available:
 1. Fit an initial transformation (doesn't need to be very good).
 2. Assign each point in set 1 to its nearest neighbor in set 2.
 3. Estimate a transformation using these pairs.
 4. Transform the points in set 1 using the estimated transformation.
 5. Repeat step 2-4 until convergence.